

Reinforcement Learning for Multi-agent Patrol Policy

Zhaohui Hu, Dongbin Zhao, *member, IEEE*

Abstract—This paper presents a reinforcement learning (RL) algorithm for multi-agent patrol tasks, which can be thought of as a dynamic programming problem with stochastic demands. We define the *cover rate* as the reward, the multi-agent physical positions including edges and nodes as the state, and the nodes adjacent to the agent as the action to model the patrol task. The modeling of this problem is totally different from other's work, which facilitates the communication and cooperation among these agents. Furthermore, we map the state from four dimensions to one dimension in order to improve the training efficiency and reduce the coding complexity. A deterministic Softmax algorithm is designed for comparison. We test both two algorithms in patrolling and rescuing scenarios. Results show the patrol *cover rate* with RL greatly outperforms Softmax about 15.38%, and the *average rescue time* for emergent pots is reduced by 20% with RL compared to Softmax.

I. INTRODUCTION

REINFORCEMENT learning is widely used to achieve the optimal control or to find the optimal policy. In RL, the agent learns from iterative trials and errors to achieve the goal [1]. Somehow we always need multi-agent to achieve the goal because the control system is more and more complex and the multi-agent approach can always be more efficient than the simply combination of them [2][3]. However, how to coordinate and cooperate between these agents is always hard to solve and the result is not satisfactory.

Patrol problem can be deemed as one of the multi-agent cooperation problems, to travel a graph with the purpose of achieving the best patrol effect. The patrol problem is a typical problem and has broad application prospects. It can not only be used in real patrol problems but also in web inspection [4], giving instructive information for ambulance vehicles, finding people in dangerous situation that need help from rescue robots and so on [5].

Some research groups did some pioneer work on the patrol task, such as the theoretical analysis of multi-agent patrolling strategies by Chevaleyre *et al.* [6], and the graph patrolling and partitioning with multi-agent algorithm by Elor *et al.* [7]. Some approaches were introduced, such as pioneer agents approach, TSP based single cycle approach, heuristic agents approach, and negotiation mechanisms for patrolling [8] and so on.

Some RL approaches were also applied to the patrol task

by Santana *et al.* [5]. In their work, the reward was defined as the idleness of each node; the state was defined by the current position of the agent as well as the idleness of each node, and the probability of the state's transition matrix was proved to be deterministic. It was proved that by maximizing the sum of these discounted rewards, they can get the minimized average idleness of this task approximately.

In this paper, we model the patrol task in a more intuitive way. ①: Propose the multi-agent *cover rate* as the reward. Basically, whenever there is an emergent call requirement in the patrolling area, at least one agent should rush to the emergent pot. As a result, the agent should cover as much area as possible during the process of patrolling. ②: The state of RL is represented by the physical location of these agents. ③: The agent action of RL is defined as its adjacent pots. ④: The transition matrix is proved to be deterministic.

After the modeling of this problem, a simulation platform for patrolling and rescuing called SEVRPSS (Simulation of Emergency Vehicles and Rescue Patrol Scheduling Software) is developed to monitor these agents' movements under this intuitive definition. To reduce the coding complexity as well as to speed up the learning efficiency of RL, a mapping algorithm is introduced to transform the state from four dimensions to one dimension. At last, we design a Softmax algorithm to compare the performances of these two algorithms.

This paper is organized as follows: Section 2 gives a description and discussion of the patrol task; Section 3 designs the reinforcement learning approach and gives the mapping method for the patrol task; Section 4 designs the Softmax algorithm approach; Section 5 carries out the experiments including both the patrolling and rescuing scenarios. Section 6 presents the discussion and conclusion.

II. THE PATROL PROBLEM AND MAP INFORMATION

As mentioned above, there are lots of demands derived from the multi-agents problem, such as web inspection, emergence vehicles routing, robot formation task, etc. In this paper, we take the vehicle patrol problem for example to illustrate the general principle and solutions.

The map which comes from the real world represents the major area of the Second Ring road and the Chang An street in Beijing, China. There are 35 junctions and 56 links in the map shown in Fig. 1. Each junction or link is indexed by an inclusive number. The node in the figure represents the junction and the length of the edge represents the length of the link.

This work was supported by National Natural Science Foundation of China (No. 60874043).

Zhaohui Hu and Dongbin Zhao are with the Lab of Complex Systems and Intelligence Sciences, Institute of Automation, Chinese Academy of Sciences, Beijing 100190 CHINA (edwalzhz@163.com, dongbin.zhao@ia.ac.cn)

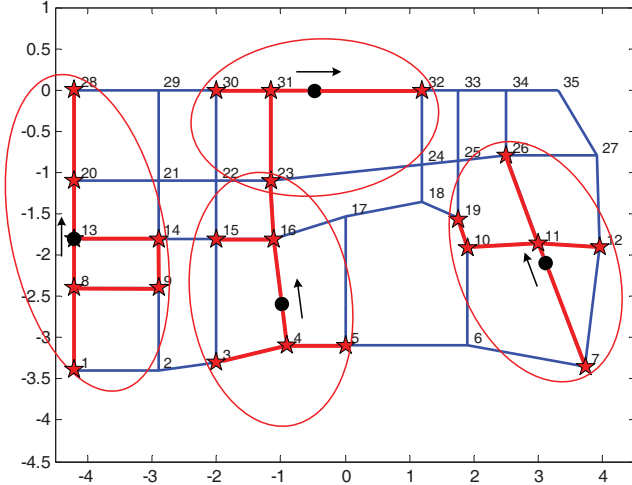


Fig. 1. The map of the patrol problem - one major area in Beijing, China. There are $n=35$ nodes and $m=56$ edges, where n is the number of the nodes, m is the number of the edges. The node represents the junction; the length of the edge represents the length of the road, the dot represents the agent, the ellipse represents the *cover range* of one agent and these pentagrams are the nodes within the *cover range*.

In patrolling, these agents travel at the speed of 40km/h, whenever there is a call requirement from any pot in the map, at least one agent should rush to the calling pot as soon as possible. So there are two main aspects to measure the patrol efficiency, which are ①: These agents should cover as many nodes as possible during the process of patrolling; ②: The agent should go to the emergent pot as soon as possible.

We define the *cover rate* as C_R to evaluate the patrol effect.

$$C_R = \text{the number of } \left(\bigcup_{i=1}^{A_N} CN_i \right) \quad i \in \{1, 2, \dots, A_N\} \quad (1)$$

where CN_i represents the nodes set within the *cover range* of the i^{th} agent (the *cover range* is the distance that an agent can reach in three minutes at the patrol speed), C_R is the number of elements of the union set of these nodes, and A_N is the number of these agents. Here we set A_N to be 4, which fits the problem size, because 5 agents will bring too many *states* to bear the computation cost, and 3 agents can hardly achieve a good patrol performance for the task.

We define the *average rescue time* as ART to evaluate the emergency response ability.

$$ART = \frac{\sum_{i=1}^{ET} T_i}{ET} \quad (2)$$

where T_i is the minimal time that the nearest agent should spend to reach the emergent pot, and ET is the number of emergent pots.

We randomly generate these emergent pots in the map at a specified time step, and then for each emergent pot, we calculate the minimal time T_i , and at last we gain the *average rescue time* at this specified time step.

III. REINFORCEMENT LEARNING APPROACH

A. Reinforcement Learning Scheme

The scheme of reinforcement learning is shown in Fig. 2. The agent gains its experience from the interaction with the environment: First, the agent takes an action a_t under some specific policy π . Second, the environment gives a reward r_{t+1} to the agent and it transfers to the next state S_{t+1} . Third, the agent learns from the reward, by adjusting its policy to maximize the accumulative rewards. The circulation of these three steps builds up the training process of RL and leads to the optimal policy π^* at last.

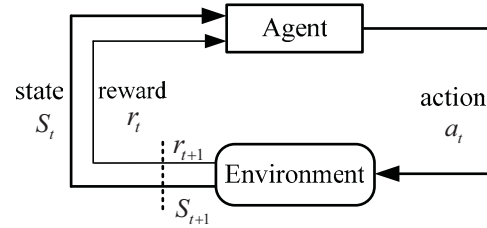


Fig. 2. Reinforcement Learning's scheme. Action a_t is taken according to the agent; the environment gives a reward r_{t+1} to the agent, and shifts the state from s_t to s_{t+1} . The agent learns its experience from the reward.

B. Temporal-Difference Learning: Q-learning

In order to gain the optimal policy, we need to define some measurable variables to distinguish which states are better, so we define the sum of discounted consequence rewards as the *Return* R_t by

$$R_t = r_{t+1} + \gamma r_{t+2} + \gamma^2 r_{t+3} + \dots = \sum_{k=0}^T \gamma^k r_{t+k+1} \quad (3)$$

where $0 \leq \gamma \leq 1$ represents the discount factor, t represents the time step, and the T is the terminal state.

Due to the stochastic property of the tasks, we use the expected *Return* to take place of the *Return*. Therefore, we have the state *value function* $V^\pi(s)$ for policy π as

$$V^\pi(s) = E_\pi \{ R_t \mid s_t = s \} \quad (4)$$

Due to the Markov property of the model, we hardly use (4). Therefore, we extend it as follows:

$$V^\pi(s) = \sum_a \pi(s, a) \sum_{s'} P_{ss'}^a \left[\mathcal{R}_{ss'}^a + \gamma V^\pi(s') \right] \quad (5)$$

where $\pi(s, a)$ is the probability to choose action a at the state s , $P_{ss'}^a$ is the probability of shifting from the state s to s' after taking action a , and $\mathcal{R}_{ss'}^a$ is the reward gained from the state shifting after taking action a .

Because of the demanding requirement of the model (including $\pi(s, a)$ and $P_{ss'}^a$), the *value function* can be hardly gained through (5). Therefore, it is usually calculated by

$$V(s) = V(s) + \alpha [r + \gamma V(s') - V(s)] \quad (6)$$

where α is the step size parameter, and this one-step temporal difference learning method requires no prior

knowledge of the environment's dynamics. Similarly, we have the state-action value function

$$Q(s, a) = Q(s, a) + \alpha[r + \gamma \max_{a'} Q(s', a') - Q(s, a)] \quad (7)$$

In each training episode, we update the current action-state value with the next maximum action-state value at every time step. We repeat this procedure until the action-state value converges, and at the same time we gained the optimal policy π^* . This is the Q -learning algorithm firstly proposed by Watkins [10].

C. Reinforcement Learning For Patrolling

There are four main elements in the reinforcement learning model: the action, the state, the reward and the transition matrix. After defining these four elements, the reinforcement learning model for the patrol task is established.

Let's turn back to the process of patrolling. Each agent patrols from one node to another node with the purpose of achieving the best patrolling performance. When one agent reaches the node N_t , with other agents in edge $(E1_t, E2_t, E3_t)$, there are several optional actions for him to choose from (if there are two more agents on the nodes at the same time, we choose the agent on the node randomly, other agents/agent on the nodes/node can be seemed as in the edge, which is because a node is surely in the edge). After one action is taken, there must be an agent (maybe the previous agent) arriving at the next node firstly. These steps make up the patrol process.

1) *The state*: The first step to shape the model may be the definition of the state. There are several principles for a good definition of the state. First, the state had better been defined to describe the process as MDP (Markov Decision Process); although Q -learning has no requirement for MDP, somehow, MDP will simplify the problem. Second, the "curse of dimensionality" is always a big problem in RL; we had better avoid it by a smart definition of the state.

From the patrolling process described in Fig. 3 and Fig. 4, we can define the state in an intuitive way: the combination of one agent on the node and three other agents in the edge $S_t = (N_t, E1_t, E2_t, E3_t)$. If there are two more agents on nodes, we randomly choose one agent on the nodes to be N_t , and the other agents in the edge $(E1_t, \dots)$ which is because the agents on the nodes must be in the edges. We have several reasons for this definition: the state is the physical position of these agents; it is easy to transfer the state to the next; the action is only decided by the current state.

Since the state consists of 4 elements, it will bring high complexity and low efficiency for the tabular Q -learning. As a result, mapping method is introduced to mirror the state from four dimensions to one dimension shown in Fig. 5. We use the rules below to construct the mapping algorithm:

①: One node index and three edge indexes make up the primary image, and a single index composes the mirror image.

②: The order of the edge indexes is ignored, that is to say, the combination of (1, 2, 3, 4) is the same as (1, 3, 4, 2).

③: No more than two agents can be in the same edge, because it will lead to definitely low *cover rate*, e.g., the combination of (1, 2, 3, 2) is not allowed.

④: The permutation of the node index (x) and the edge combination index (y) determine the final mapping index. Just as shown in Fig. 5, $x + y$ makes the final mapping result.

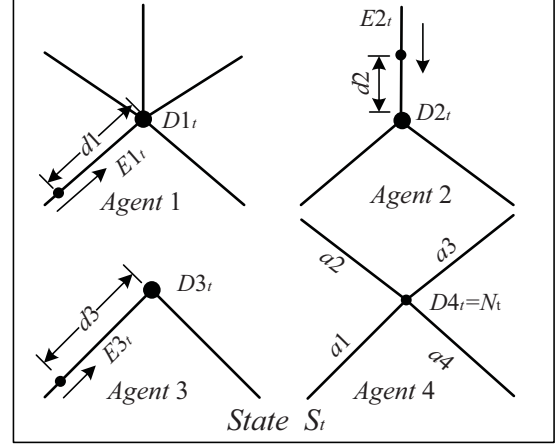


Fig. 3. The definition of *state*, *direction*, *action*, *remaining distance*, *pointer*. The *state* is the combination of a node and three edges, the *direction* are these nodes that agents are heading for separately, the *action* set is the adjacent nodes next to the agent, the *remaining distance* is the distance from the agent to its *direction*. Here the *state* is $(E1_t, E2_t, E3_t, N_t)$, the *direction* is $(D1_t, D2_t, D3_t, D4_t)$, the *action* is $(a1, a2, a3, a4)$, and the *remaining distance* is $(d1, d2, d3, 0)$.

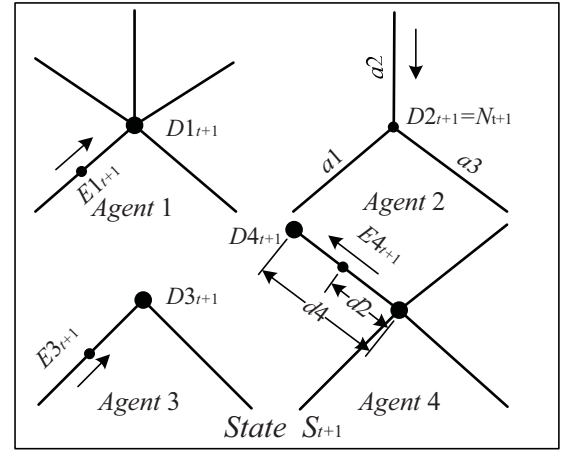


Fig. 4. The state S_{t+1} , after taking action $a2$, agent 1 and agent 3 are closer to their direction by the length $d2$, agent 2 arrives at the node N_{t+1} firstly, and agent 4 changes the direction from $D4_t$ to $D4_{t+1}$. So the *state* changes to $(E1_{t+1}, N_{t+1}, E3_{t+1}, E4_{t+1})$, the *direction* changes to $(D1_{t+1}, D2_{t+1}, D3_{t+1}, D4_{t+1})$, the *action* is $(a1, a2, a3)$, and the *remaining distance* changes to $(d1 - d2, 0, d3 - d2, d4 - d2)$.

⑤: The permutation of the node index is calculated with (8). For each node, there are C_m^3 combinations. That is why the multiplication operator is used, and as a result, the total combination is $n \cdot C_m^3$.

$$x = (N_t - 1) \cdot C_m^3 \quad (8)$$

where m is the number of edges in the map, and N_t is the node

that the agent is on (the state is $S_t = (N_t, E1_t, E2_t, E3_t)$).

⑥: The combinational index of the three edges is calculated with (9). It is easy to validate that the combination (1, 2, 3) is indexed by 1, and (54, 55, 56) is index by C_{56}^3 .

$$y = \sum_{t=1}^{E1_t-1} C_{m-t}^2 + \sum_{t=2}^{E2_t-1} C_{m-t}^1 - \sum_{t=2}^{E1_t} C_{m-t}^1 + \sum_{t=3}^{E3_t-1} C_{m-t}^0 - \sum_{t=3}^{E2_t} C_{m-t}^0 \quad (9)$$

where n is the number of nodes in the patrol map. It is easy to conclude that the assumptions $E1_t < E2_t < E3_t$ is reasonable from rules ② and ③.

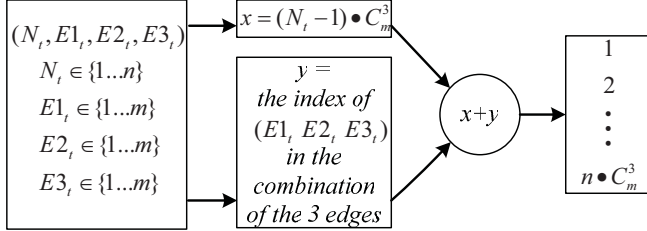


Fig. 5. State mapping from 4 dimensions to 1 dimension, n is the number of nodes; m is the number of edges.

2) *The action:* After the definition of the state, the action definition is easily.

When the agent is in state S_t , there are several nodes ($a1, a2, a3, a4$) that are adjacent to the node N_t shown in Fig. 3 (The action number will vary with various node structures). The agent on node N_t chooses one action according to some specified policy. In this paper, epsilon greedy policy is adopted and defined by

$$a = \begin{cases} \arg \max_a Q^\pi(s, a) & \text{with probability } 1 - \varepsilon \\ \text{random action} & \text{with probability } \varepsilon \end{cases} \quad (10)$$

where $Q^\pi(s, a)$ is the action-state value, and ε is a small number.

3) *The transition matrix:* The transition probability is deterministic.

We define the *direction* $dir = (D1_t, D2_t, D3_t, D4_t)$ to be nodes that these agents are heading for separately, the *remaining distance* $dis = (d1, d2, d3, d4)$ to be the distances between the agent's positions and their directions separately.

When the agent is in state S_t shown in Fig. 3, the *remaining distance* is $dis = (d1, d2, d3, 0)$, and the *direction* is $dir = (D1_t, D2_t, D3_t, N_t)$ (the direction of the agent on the node is set to be N_t). If there are one more agents reaching these nodes at the same time, we choose the agent on the node randomly, leaving the other agents on the node with the *remaining distance* di to be zero.

After the agent takes an action, such as $a2$ shown in Fig. 4, the *remaining distance* changes to $dis = (d1, d2, d3, d4)$ immediately, and the *direction* changes to $dir = (D1_{t+1}, D2_{t+1}, D3_{t+1}, D4_{t+1})$. There must be a minimal length in the *remaining distance* and we assume it to be $d2$, as a result, agent 2 will be on the node at the $t+1$ time step.

After some time when the state shifts to S_{t+1} , the *direction* is still $dir = (D1_t, D2_t, D3_t, D4_t)$, but the *remaining distance* changes to $dis = (d1-d2, 0, d1-d2, d4-d2)$.

From the discussion above, it is obvious that the next state is surely led to a deterministic state after taking an action just like the process shown in Fig. 4.

4) *The reward:* The reward definition should reflect the purpose of the patrol task; otherwise, the result will betray the user's intent.

When the state is $S_t = (N_t, E1_t, E2_t, E3_t)$ with the *remaining distance* $dis = (d1, d2, d3, 0)$, we can use these information to get the number of nodes that are in the *cover range*, just as those pentagrams shown in Fig. 1. As a result, the reward is defined by (11)

$$r = C_R - C \quad (11)$$

where C_R is defined above in (1), C is a constant number to emulate the half of the average *cover rate*, which is set 13 after some trials, and the minus operator is introduced to give a penalty to the low *cover rate*. It can be concluded from the definition that the higher the reward is; the better patrol effect will be gained, *vice versa*, which undoubtedly reflects the purpose of patrolling.

IV. SOFTMAX APPROACH

Now, we shape the task into another model: Softmax model, and then compare it with the RL model.

In this model, the state, action, reward are defined the same as these in RL, and the state transition probability matrix is also deterministic. But the action is decided by the Softmax algorithm, and there is no tabular Q value, not to say updating the value function.

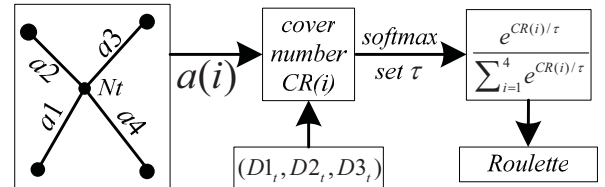


Fig. 6. The action decision process. The *cover rate* is mapped to the probability of selection.

The action decision process is shown in Fig. 6. When an agent is on the node N_t , there are several directions for it, such as ($a1, a2, a3, a4$). Each action $a(i)$ as well as other directions ($D1_t, D2_t, D3_t$) have a corresponding *cover rate* $CR(i)$. $CR(i)$ is set to be zero manually when the combination is the same as the previous state because returning back is deemed as bad performance. According to (12), *cover rate* is transformed to the probability of selection.

$$p(i) = \frac{e^{CR(i)/\tau}}{\sum_{i=1}^u e^{CR(i)/\tau}} \quad (12)$$

where u represents the optional actions number that the agent has, τ represents the temperature, and $p(i)$ is the selection probability of the i^{th} option.

The probability of the selection tends to be equal when the temperature is high, and has a great difference when the temperature is low. When properly setting the temperature, it can distinguish the *cover rate* significantly, e.g., when the temperature is 3, the *cover rate* changes from (18, 20, 21, 0) which is quite usual in this task to (0.1764, 0.3436, 0.4795, 0.0004). As a result, it can distinguish these actions' priority significantly.

After the probability calculation, a Roulette algorithm is used to choose an action, and then a reward is given.

V. SIMULATION RESULTS

In the simulation part, we compare the patrolling and the rescuing performance of these two algorithms.

These experiments are carried out on the platform of SEVRPSS (Simulation of Emergency Vehicles and Rescue Patrol Scheduling Software) that can be used for tracking the performance of these agents and analyzing the results. The main interface is shown in Fig. 7.

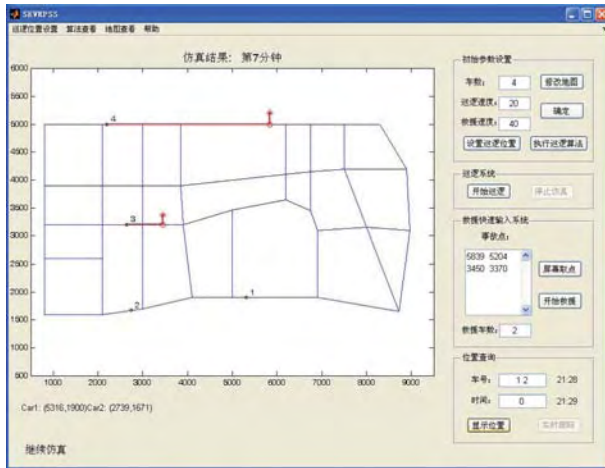


Fig. 7. Patrolling and rescuing simulation platform. This platform can be used for animation of these agents' position for both the patrolling and rescuing. The red line represents the optimal path that these agents should take to the emergent pot. The small dots represent these agents. The right side of this interface can be used for setting the patrolling and rescuing speed, changing the map, setting the initial agents' positions, setting the emergent pots' positions, and inquiring the agents' positions. The menu bar of this platform can be used for inquiring the map, changing the patrolling and rescuing algorithms and so on.

A. Patrolling Experiment

In the reinforcement learning model, the *discount factor* γ and the *step size parameter* α are both set to be 0.9. The *epsilon* is decreased by $\delta = 0.000001$ at each training episode from 0.2 to 0.1, with the purpose of leading the agent to a better choice.

As mentioned above, we define the number of states $n \cdot C_m^3 = 970200$ to be N . In the RL experiments, we run $250N$ time steps and record these rewards gained during the process. In each N time steps, we plot its maximal reward, minimal reward and average reward.

The results of RL is shown in Fig. 8: the maximal reward increases to the optimal value 22 after about $15N$ time steps, the minimal reward increases from about -4 to 0 during the whole process, and the average reward increases from 10 to 17 in the beginning $50N$ time steps and maintains this value, as a result, the average *cover rate* for RL is 30 according to (11). It is inferred that these fluctuations with the minimal reward are caused by the stochastic property of the epsilon greedy action chosen algorithm.

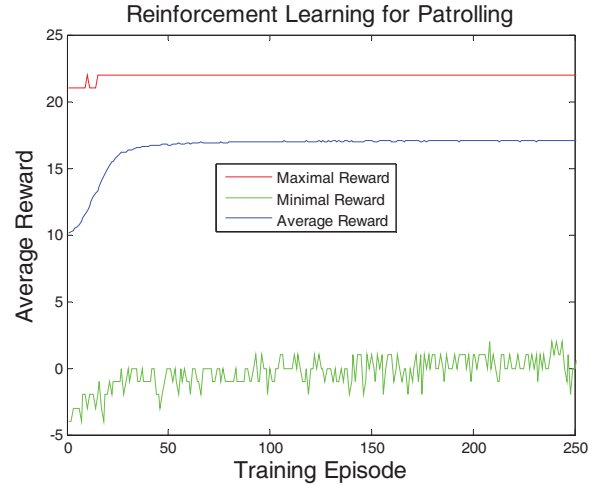


Fig. 8. Reinforcement learning performance for patrolling

In the Softmax experiment, we run $100N$ time steps and record these rewards gained during the process. The results of the Softmax algorithm show that the maximal reward gained is 19, the minimal reward gained is about -2, and the average reward gained is 13 all the time. As a result, the average *cover rate* is 26 according to (11). Moreover, there is not any hint that the performance will be better, so we don't run $250N$ time steps. It is inferred that the fluctuations of the minimal reward are caused by the stochastic property of the Softmax algorithm.

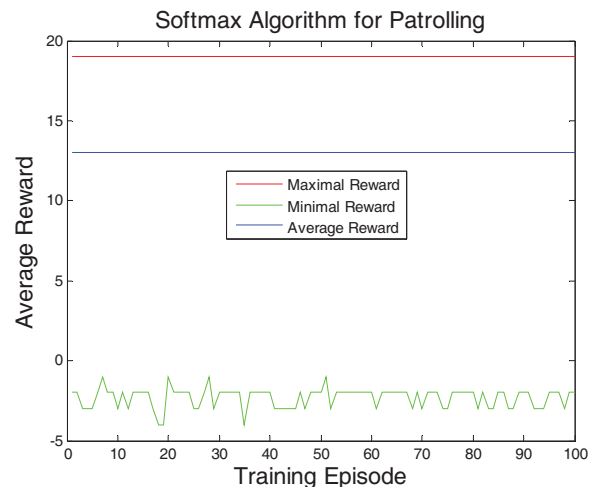


Fig. 9. Softmax algorithm performance for patrolling

B. Rescuing Experiment

After the patrolling experiment, we also have the rescuing experiment. We test the *average rescue time* for the emergence case with these two algorithms.

At the time step 30, we randomly generate 20 emergent pots, we plot the time the nearest agent should taken to the emergent pot at each emergent trail shown in Fig. 10. Because of the random property of the generated emergent pots and the different agents' positions between the two policies, the *average rescue time* with Softmax is shorter in some cases. However, at most of the time, the minimal time with RL is lower than that with Softmax.

Also, we calculate the average time for both algorithms. The *average rescue time* for Softmax is 201.9620s, and the *average rescue time* for RL is 159.5580s which is 20% shorter.

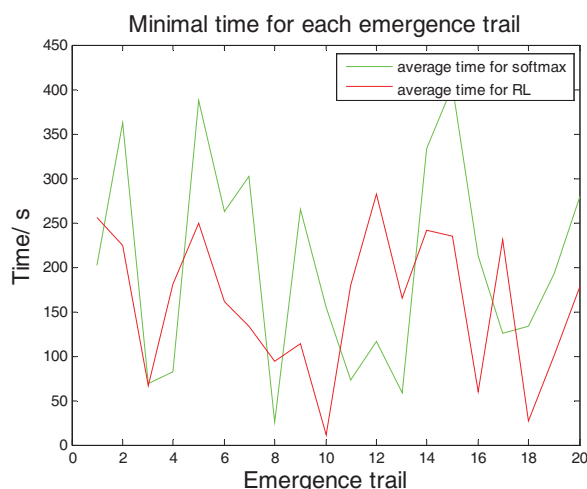


Fig. 10. Minimal time for each emergence trail with both the RL and Softmax algorithm.

VI. DISCUSSION AND CONCLUSION

We can draw the following three conclusions from Fig. 8 and Fig. 9:

①: At the early stage of training, the average reward with RL is lower than the Softmax approach. This is because the Q value is initialized to be zero in RL, which means no experience for these agents in the beginning. While in the Softmax approach, the action is decided by the Softmax algorithm, which means that higher expected *cover rate* is more likely to be obtained. As a result, RL doesn't perform better than Softmax in the beginning.

②: The maximal reward gained by RL is always higher than that with Softmax, and the minimal average gained by RL is better than that gained by Softmax after some training.

③: The average reward increases with the training process. At the $25N$ time steps, the average reward with RL begins to catch up with Softmax, while there isn't any hint for the increasing trend of the Softmax approach. At the $50N$ time step, the average reward comes to its convergent value 17,

while the Softmax approach maintains its convergent value 13. As a result, the *cover rate* is 30 and 26 for RL and Softmax separately, and the *cover rate* is 15.38% higher than Softmax.

From Fig. 10, we can conclude that the RL method can not only has a good performance in the patrolling scenario, but also in the rescuing scenario, and the average time for rescuing is reduced by 20% compared with the Softmax algorithm.

Therefore, we can draw the conclusion safely that these agents can be effectively organized in the patrol task based on the proposed RL algorithm.

REFERENCES

- [1] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*. Massachusetts London, England: The MIT Press Cambridge, 1998.
- [2] J. Ferber, *Multi-Agent System: An Introduction to Distributed Artificial Intelligence*, Boston, MA: Addison-Wesley Longman Publishing Co., Inc., 1999.
- [3] W. Shang, and D. Sun, "distributed neural network-based policy gradient reinforcement learning for multi-robot formations," *Proceedings of the International Conference on Information and Automation, Zhangjiajie, China*, June 20-23, 2008, pp.113-118.
- [4] R. C. Andrade, H. T. Macedo, G. L. Ramalho, and C. A. G. Ferraz, "Distributed mobile autonomous agents in network management," *Proceedings of International Conference on Parallel and Distributed Processing Techniques and Applications*, 2001.
- [5] H. Santana, G. Ramalho, V. Corruble, and B. Ratitch, "Multi-agent patrolling with reinforcement learning," *Proceedings of the Third International Joint Conference on Autonomous Agents and Multiagent Systems*, 2004, pp.1122-1129.
- [6] Y. Chevaleyre, F. Sempé, G. Ramalho, "A theoretical analysis of multi-agent patrolling strategies," *Proceedings of the Third International Joint Conference on Autonomous Agents and Multiagent Systems*, Vol. 3, July 19-23, 2004, New York, USA. pp.1524-1525.
- [7] Y. Elor and A. M. Bruckstein. "Multi-agent graph patrolling and partitioning," *Proceedings of the 2009 IEEE/WIC/ACM International Joint Conference on Web Intelligence and Intelligent Agent Technology*, 2009, pp. 52-57.
- [8] A. Almeida, G. Ramalho, H. Santana, *et al.*, "Recent Advances on Multi-agent Patrolling," in A. L. C. Bazzan and S. Labidi, eds, *SBIA 2004, LNAI 3171*, Springer-Verlag Berlin Heidelberg, pp.474-483. 2004.
- [9] R. W. Floyd, Algorithm 97 (shortest path). *Communications of the ACM*, 1962, vol.5, no.6, pp.345.
- [10] C. Watkins, "Q-learning," *Machine Learning*, 1992, vol. 8, no.3, pp.279-292.
- [11] A. G. Barto, T. G. Dietterich, "Reinforcement learning and its relationship to supervised learning," in J. Si, A. Barto, W. Powell, and D. Wunsch. *Handbook of Learning and Approximate Dynamic Programming*, IEEE Press, John Wiley & sons, Inc., 2004, pp. 47-63.
- [12] M. T. Rosenstein and A. G. Barto, "Supervised actor-critic reinforcement learning," in J. Si, A. Barto, W. Powell, D. Wunsch. *Handbook of Learning and Approximate Dynamic Programming*, IEEE Press, John Wiley & sons, Inc., 2004, pp. 359-380.